Software Engineering Lab

Lab 29

Saga Pattern: Transaksi Terdistribusi Tanpa 2PC

Eventual Consistency melalui LIFO Compensation & Semantic Locks

Lukman / lukman-ss

Keterbatasan ACID Monolitik di Dunia Microservices

Database-per-Service memecah transaksi lintas boundary layanan mandiri

MASALAH DASAR

Arsitektur microservices menuntut setiap service memiliki database sendiri
Transaksi bisnis seperti Checkout membutuhkan koordinasi Order, Payment, dan Inventory
Pola BEGIN TRANSACTION...COMMIT tunggal tidak dapat melintasi batas jaringan database

KEGAGALAN TWO-PHASE COMMIT (2PC)

2PC mengunci baris data di seluruh node database selama fase persiapan (prepare)
Koordinator tunggal menjadi single point of failure (SPOF) dan latency multiplier
Partisi jaringan membuat resource terkunci tanpa batas (blocking protocol)

Sistem beralih dari ACID ketat menuju BASE (Basically Available, Soft state, Eventual consistency)

Saga pattern memecah transaksi global menjadi rangkaian transaksi lokal independen

Setiap transaksi lokal langsung melakukan commit di databasenya masing-masing

2PC locks distributed databases synchronously; Sagas commit locally and coordinate asynchronously.

Anatomi Saga: Rangkaian Transaksi Lokal

Eksekusi maju teratur dan pembatalan mundur terarah

FORWARD EXECUTION

T1 (Order Created) -> T2 (Payment Processed) -> T3 (Inventory Reserved) -> T4 (Order Approved)
Setiap step mengeksekusi mutasi data lokal dan mencatat status EXECUTED
Jika semua langkah berhasil, transaksi terdistribusi selesai secara konsisten

COMPENSATION ROLLBACK (LIFO)

Jika T3 gagal (misal out-of-stock), orchestrator memicu kompensasi
Urutan mundur: C2 (Refund Payment) -> C1 (Cancel Order)
Prinsip tumpukan LIFO menjamin efek dibatalkan dalam urutan terbalik

Kompensasi adalah transaksi bisnis baru, bukan pembatalan level storage engine
Setiap langkah menghasilkan log audit permanen di sistem
Rollback tidak menghapus record masa lalu melainkan menuliskan saldo koreksi

A saga is a chain of local commits; failure triggers a reverse chain of business compensations.

Tiga Jenis Transaksi dalam Saga

Klasifikasi formal langkah transaksi berdasarkan kemampuan kompensasi

1. COMPENSABLE TRANSACTIONS

Langkah yang dapat dibatalkan atau dikoreksi dengan aksi berlawanan
Contoh: CreateOrder (dibatalkan oleh CancelOrder), ChargeCard (RefundPayment)
Wajib memiliki pasangan kompensasi eksplisit sebelum titik balik tercapai

2. PIVOT TRANSACTION

Titik balik tanpa kembali (point of no return) dalam proses bisnis saga
Jika transaksi pivot berhasil, saga dijamin akan terus berjalan hingga tuntas
Jika transaksi pivot gagal, seluruh langkah compensable sebelumnya harus di-rollback

3. RETRYABLE TRANSACTIONS

Langkah yang dieksekusi setelah pivot dan dijamin pada akhirnya akan berhasil

Contoh: Kirim notifikasi email, approval final order, pembaruan analitik

Wajib bersifat idempotent karena dapat di-retry berulang kali oleh worker

Compensable before pivot, pivot decides the fate, retryable guarantees completion after pivot.

Dua Pola Koordinasi: Orchestration vs Choreography

Sentralisasi kendali alur versus desentralisasi reaktif berbasis event

ORCHESTRATION (CENTRALIZED)

Koordinator sentral memegang state machine dan memanggil service satu per satu
Alur bisnis eksplisit, mudah di-audit, dan penanganan LIFO rollback terpusat
Kelemahan: Risiko kopling logika bisnis ke satu koordinator jika tidak dirawat

CHOREOGRAPHY (EVENT-DRIVEN)

Setiap service mempublikasikan domain event dan bereaksi terhadap event service lain
Tingkat decoupling tinggi tanpa orchestrator tunggal di tengah
Kelemahan: Alur implisit, sulit melacak state global, risiko siklus event tak berujung

Gunakan Orchestration untuk workflow kompleks dengan >3 langkah dan kompensasi rumit

Gunakan Choreography untuk integrasi sederhana 2–3 service yang minim ketergantungan balik

Lab 29 mengimplementasikan kedua model untuk perbandingan langsung yang objektif

Orchestration gives explicit central control; Choreography gives loose event-driven autonomy.

Arsitektur Lab 29: Komponen & Boundary

Pemisahan domain services, state machine orchestrator, dan decoupling event bus

SAGA CORE (`INTERNAL/SAGA`)

Orchestrator: Mengelola slice Step, eksekusi sekuensial, dan kompensasi LIFO
StepLog: Mencatat status PENDING, EXECUTED, FAILED, COMPENSATED, COMPENSATE_FAILED
EventBus: Memfasilitasi publish-subscribe berbasis thread-safe sync.RWMutex

DOMAIN SERVICES (`INTERNAL/SERVICES`)

OrderService: Mengelola status pesanan dan semantic lock untuk pencegahan race condition
PaymentService: Memproses pembayaran dengan idempotency key per payment ID
InventoryService: Reservasi dan pelepasan stok inventaris barang

cmd/demo/main.go: Mendemonstrasikan Happy Path dan Rollback Out-of-Stock di CLI

tests/saga_test.go: 9 skenario pengujian komprehensif termasuk concurrency race test

Zero external dependencies — murni menggunakan Go standard library untuk akurasi tertinggi

Clean component separation isolates domain logic from distributed transaction coordination.

Implementasi Orchestrator & Struktur Step

Fungsi Execute dan Compensate sebagai kontrak utama setiap tahapan

```
type Step struct {
    Name       string
    Execute    func(ctx context.Context) error
    Compensate func(ctx context.Context) error
}
Tracking status: PENDING, EXECUTED, FAILED, COMPENSATED, COMPENSATE_FAILED
```

```
type Orchestrator struct {
    mu    sync.Mutex
    steps []Step
    logs  []StepLog
}
Setiap mutasi log dilindungi mutex untuk mendukung akses paralel
```

Compensate bersifat opsional untuk langkah read-only atau pivot murni

Jika Compensate bernilai nil, langkah tersebut dilewati tanpa error saat rollback

Menjamin fleksibilitas dalam memodelkan ragam jenis transaksi bisnis

Every orchestrator step pairs a forward action with an optional reverse compensation.

Algoritma Kompensasi LIFO: Membalik Riwayat

Eksekusi mundur dari langkah terakhir yang sukses menuju langkah pertama

```
func (o *Orchestrator) compensate(ctx context.Context, executed []Step) error {
    var compErrors []error
    for i := len(executed) - 1; i >= 0; i-- {
        step := executed[i]
        if step.Compensate != nil {
            err := step.Compensate(ctx)
            // append error or mark COMPENSATED...
        }
    }
    return aggregateErrors(compErrors)
}
```

Langkah belakangan seringkali bergantung pada status yang dibuat langkah sebelumnya

Contoh: Dana harus dikembalikan (refund) sebelum pesanan dibatalkan (cancel)

Eksekusi acak atau paralel berisiko melanggar batasan konsistensi domain

Kompensasi dipanggil menggunakan context.Background()

Tujuannya mencegah pembatalan context request induk mematikan proses pembersihan data

Menjamin proses rollback tetap berjalan meski client melakukan disconnect

Unwind executed steps in reverse order (LIFO) so dependent actions undo gracefully.

Event-Driven Choreography via EventBus

Koordinasi desentralisasi melalui publish/subscribe antar domain service

```
type EventBus struct {
    mu       sync.RWMutex
    handlers map[EventType][]EventHandler
}
Mendukung pendaftaran banyak subscriber untuk satu event type
```

ALUR EVENT CHOREOGRAPHY

> OrderCreated -> diproses PaymentService -> PaymentCompleted dipublish
> PaymentCompleted -> diproses InventoryService -> InventoryReserved dipublish
> InventoryReserved -> diproses OrderService -> OrderApproved dipublish

Jika reservasi inventaris gagal -> InventoryFailed dipublish
PaymentService mendengar InventoryFailed -> melakukan refund pembayaran
OrderService mendengar PaymentFailed/InventoryFailed -> membatalkan pesanan

Choreography lets services react to domain events without knowing who triggered them.

Semantic Locking: Melawan Anomali Tanpa Isolasi ACID

Menandai record sebagai PENDING untuk mencegah modifikasi konkuren yang merusak

```
if s.locks[orderID] {
    return errors.New("order is semantically locked")
}
s.orders[orderID] = OrderPending
s.locks[orderID] = true
```

Karena transaksi lokal langsung di-commit, transaksi lain dapat membaca data sementara

Jika saga kemudian di-rollback, data yang dibaca transaksi lain menjadi invalid

Saga tidak memiliki isolasi level database bawaan antar service

Kunci dilepas saat saga mencapai status terminal (ApproveOrder atau CancelOrder)

Request lain yang mencoba memodifikasi pesanan selama pending akan ditolak

Mencegah overwriting data yang sedang dalam proses verifikasi pembayaran

Semantic locks flag uncommitted business state to prevent concurrent dirty modifications.

Idempotency Keys: Fondasi Retry yang Aman

Menjamin pemanggilan ulang akibat network glitch tidak menduplikasi mutasi dana

```
func (s *PaymentService) ProcessPayment(id string, amt float64, ...) error {
    s.mu.Lock(); defer s.mu.Unlock()
    if s.processedID[id] {
        return nil // Idempotent success: lewati eksekusi
    }
    // proses pembayaran baru...
}
```

Saat memanggil PaymentService, timeout jaringan dapat terjadi setelah dana terpotong

Orchestrator tidak tahu apakah request berhasil atau gagal sehingga wajib melakukan retry

Tanpa proteksi, retry akan memotong saldo pelanggan dua kali

Kunci idempotensi wajib disertakan pada semua Compensable dan Retryable transactions

Memungkinkan worker mencoba kembali berkali-kali hingga mencapai kepastian status

Menjaga konsistensi data finansial di tengah ketidakpastian jaringan terdistribusi

Idempotency turns repeated network retries into safe no-ops without double execution.

Manajemen Konkurensi & Pembatalan Konteks

Deteksi ctx.Done() dan perlindungan terhadap race condition

```
select {
case <-ctx.Done():
    o.logs = append(o.logs, StepLog{Name: step.Name, Status: StatusFailed})
    compErr := o.compensate(context.Background(), executed)
    return fmt.Errorf("saga cancelled: %w", ctx.Err())
default:
}
```

Setiap service (Order, Payment, Inventory) memiliki sync.Mutex mandiri
Saga Orchestrator memproteksi slice log dan steps dari konkurensi
Menjamin integritas internal state saat diakses oleh banyak goroutine
10 transaksi saga dijalankan bersamaan dalam goroutine terpisah
Setiap saga mengelola pesanan unik dengan lock dan state lokalnya sendiri
Lulus pengujian go test -race tanpa terdeteksi satu pun data race condition

Check context before every step and execute compensations with an uncancelled context.

Eksekusi Demo: Skenario Happy Path

Seluruh langkah bisnis tuntas dan disetujui tanpa hambatan

```
=== Scenario 1: Happy Path (Orchestrator) ===
 -> [OrderService] Creating order: ord-success
 -> [PaymentService] Processing payment for: ord-success
 -> [InventoryService] Reserving 1 laptop
 -> [OrderService] Finalizing order approval
Scenario 1 Result: error=, OrderState=APPROVED, Remaining Stock=0
```

OrderService mencatat orderState = APPROVED

PaymentService menyimpan record transaksi senilai 1500.0

InventoryService mendegradasi stok laptop dari 1 menjadi 0

Seluruh 4 langkah mencatat status EXECUTED

Tidak ada fungsi kompensasi yang dipanggil selama proses berjalan

Sistem mencapai eventual consistency dalam hitungan milidetik

When all steps succeed sequentially, the saga completes and leaves domain states committed.

Eksekusi Demo: Skenario Kegagalan Stok & Rollback

Out-of-stock memicu pembatalan pembayaran dan pesanan via LIFO

```
=== Scenario 2: Rollback on Failure (Orchestrator) ===
 -> [OrderService] Creating order: ord-failed
 -> [PaymentService] Processing payment for: ord-failed
 -> [InventoryService] Reserving 1 laptop (current stock: 0)
 <- [PaymentService] Compensating: Refunding payment for: ord-failed
 <- [OrderService] Compensating: Cancelling order: ord-failed
Scenario 2 Result: error=step ReserveInventory failed: out of stock
Final State: OrderState=CANCELLED, HasPayment=false, Stock=0
```

Langkah ke-3 (ReserveInventory) gagal karena stok bernilai 0

Orchestrator memutar balik: RefundPayment dipanggil terlebih dahulu

Setelah refund selesai, CancelOrder dipanggil untuk menutup pesanan

Pelanggan tidak dirugikan karena uang berhasil di-refund otomatis

Status pesanan tercatat CANCELLED bukan hilang dari database

Konsistensi bisnis pulih tanpa memerlukan kunci database global

A failed step immediately halts forward execution and triggers exact LIFO compensation.

Bukti Pengujian Otomatis: 9 Test Suites

Verifikasi formal happy path, kompensasi LIFO, idempotensi, dan error propagation

```
=== RUN   TestOrchestrator_HappyPath             --- PASS (0.00s)
=== RUN   TestOrchestrator_FailureCompensatesLIFO--- PASS (0.00s)
=== RUN   TestPayment_Idempotency                --- PASS (0.00s)
=== RUN   TestSemanticLock                       --- PASS (0.00s)
=== RUN   TestOrchestrator_Concurrency           --- PASS (0.00s)
=== RUN   TestChoreography_Flow                  --- PASS (0.00s)
=== RUN   TestChoreography_FailureCompensates    --- PASS (0.00s)
=== RUN   TestOrchestrator_CompensationErrorPropagated --- PASS (0.00s)
=== RUN   TestOrchestrator_ContextCancellation   --- PASS (0.00s)
PASS: 9/9 test suites passed in 0.428s
```

TestSemanticLock: Memastikan pencegahan order creation duplikat saat status pending

TestCompensationErrorPropagated: Memverifikasi penanganan jika refund payment gagal

TestChoreography_Flow: Memvalidasi rantai event pub-sub tanpa koordinator sentral

Seluruh pengujian lolos uji deteksi data race (`go test -race ./...`)

Test case mencakup skenario happy path, sad path, dan edge cases kegagalan jaringan

Bukti deterministik bahwa koordinasi terdistribusi bekerja sesuai spesifikasi

Unit and concurrency tests strictly validate LIFO rollback, idempotency, and semantic locks.

Ketika Kompensasi Gagal: Realitas Lapangan

Penanganan status COMPENSATE_FAILED dan strategi pemulihan sistem

KONDISI COMPENSATE_FAILED

Bagaimana jika refund gateway menghasilkan timeout atau status 500 error?
Saga tidak dapat melakukan rollback otomatis tak terbatas
Orchestrator mencatat status langkah menjadi COMPENSATE_FAILED dan agregasi error

STRATEGI MITIGASI PRODUKSI

Dead-Letter Queue (DLQ): Menyimpan pesan kompensasi yang gagal untuk proses retry terpisah
Automated Alerting: Memberi sinyal instan kepada tim engineer on-call
Idempotent Retry Worker: Mencoba kembali kompensasi dengan pola exponential backoff

Dashboard operasional untuk meninjau status saga yang mengalami kebuntuan

Customer service atau finance dapat melakukan penyesuaian manual (out-of-band)

Audit trail log langkah menjadi sumber kebenaran tunggal investigasi

When business compensation fails, isolate to DLQs and rely on idempotent retry workers or manual fixes.

Evaluasi Arsitektur: ACID vs 2PC vs Saga

Perbandingan mendalam dimensi konsistensi, ketersediaan, dan skalabilitas

Dimensi	ACID Monolitik	Two-Phase Commit (2PC)	Saga Pattern
Konsistensi	Strict Immed.	Strict Immed.	Eventual Consistency
Isolasi (I)	Didukung DBMS	Didukung DBMS	Perlu Semantic Lock
Ketersediaan	Sangat Tinggi	Rendah (Blocking)	Sangat Tinggi
Skalabilitas	Terbatas 1 DB	Buruk lintas jaringan	Linear horizontal
Kompleksitas	Sangat Rendah	Menengah	Tinggi (Kompensasi)

ACID menjamin tidak ada dirty read tanpa coding tambahan

Saga menukar isolasi instan demi mempertahankan ketersediaan sistem 24/7

Cocok untuk sistem high-throughput di mana lock contention 2PC tidak dapat diterima

Tetap gunakan ACID lokal di dalam boundary tiap microservice

Gunakan Saga secara eksklusif untuk koordinasi lintas service terpisah

Choose ACID for local boundaries; use Sagas to preserve availability across distributed systems.

Ekosistem & Framework Saga di Industri

Solusi open-source dan cloud-native untuk orchestrator produksi

TEMPORAL.IO & CADENCE

Stateful workflow engine terkemuka untuk orchestration saga skala enterprise
Menjamin eksekusi fungsi Go/Java/TypeScript tahan banting terhadap crash server
Secara otomatis mencatat event history dan menangani retry serta timeout kompensasi

AWS STEP FUNCTIONS

Serverless workflow orchestrator berbasis declarative JSON/YAML state machine
Terintegrasi langsung dengan AWS Lambda, SQS, SNS, dan DynamoDB
Mendukung visualisasi alur eksekusi grafis dan penanganan Catch/Retry terstruktur

Eventuate Tram / MassTransit untuk implementasi choreography berbasis transactional messaging

Sering dipadukan dengan Outbox Pattern (Lab 21) untuk menjamin atomilitas event publishing

Solusi tepat saat membangun arsitektur event-driven terdistribusi murni

Production sagas use workflow engines like Temporal or Step Functions to survive infrastructure crashes.

Sinergi: Saga Pattern + Transactional Outbox

Menjamin penerbitan event tidak pernah hilang saat database lokal di-commit

DUAL-WRITE PROBLEM DI SAGA

Sebuah service harus melakukan dua hal: update database lokal dan publish event ke broker
Jika database commit berhasil tapi broker pesan mati, saga akan terhenti permanen
Jika broker publish berhasil tapi database commit gagal, data menjadi tidak konsisten

SOLUSI TRANSACTIONAL OUTBOX

Simpan domain event ke dalam tabel outbox di database yang sama dalam satu transaksi ACID
Proses worker independen membaca tabel outbox dan mempublikasikannya ke message broker
Menghilangkan risiko dual-write inconsistency pada arsitektur saga choreography

Lab 21 (Transactional Outbox) memberikan fondasi pengiriman pesan andal

Lab 29 (Saga Pattern) mengorkestrasi alur transaksi bisnis tingkat tinggi

Kombinasi keduanya merupakan gold standard arsitektur microservices modern

Pair Sagas with Transactional Outbox to eliminate the dual-write problem across service boundaries.

Studi Kasus Nyata: E-Commerce Checkout

Menyusuri siklus hidup 4 tahap pesanan dari keranjang hingga pengiriman

TAHAP 1: ORDER CREATION (COMPENSABLE)

Client menekan tombol Bayar Sekarang di aplikasi
OrderService mencatat pesanan PENDING dan memasang semantic lock pada order ID
Kompensasi jika gagal di langkah lanjutan: CancelOrder

TAHAP 2 & 3: PAYMENT & INVENTORY

PaymentService memotong saldo kartu dengan idempotency key (Kompensasi: RefundPayment)
InventoryService mengalokasikan stok gudang fisik (Kompensasi: ReleaseInventory)
Langkah inventaris bertindak sebagai Pivot Transaction

Setelah stok aman, order status diubah menjadi APPROVED dan semantic lock dilepas

Email invoice dikirimkan ke pelanggan via retryable worker

Pesanan kini siap diproses oleh divisi logistik pengiriman

E-commerce checkouts balance fast client confirmation with background eventual consistency.

Checklist Kesiapan Produksi Saga

9 poin verifikasi arsitektur sebelum merilis ke lingkungan live

CHECKLIST PENERAPAN SAGA

[ ] Setiap compensable step memiliki fungsi kompensasi yang teruji
[ ] Seluruh endpoint kompensasi dan retry bersifat 100% idempotent
[ ] Semantic lock atau versioning diterapkan untuk mencegah dirty reads
[ ] Dead-Letter Queue (DLQ) disiapkan untuk kompensasi yang gagal
[ ] Log audit status langkah disimpan di storage persisten
[ ] Context timeout dan cancellation dikelola tanpa membatalkan rollback
[ ] Monitoring dan alerting aktif untuk status COMPENSATE_FAILED
[ ] Prosedur rekonsiliasi manual didokumentasikan untuk tim finance
[ ] Concurrency race detector lulus tanpa anomali data

ATURAN UTAMA

Jangan pernah mengasumsikan kompensasi akan selalu berhasil tanpa cela
Rancang sistem dengan asumsi jaringan akan gagal pada momen terburuk
Saga yang baik selalu memiliki jalur pemulihan darurat

Distributed tracing (OpenTelemetry) sangat krusial untuk melacak Saga ID lintas service

A complete production checklist covers idempotency, semantic locks, DLQs, and observability.

Batasan & Lingkup Implementasi Lab 29

Membedakan simplifikasi edukasi dengan kebutuhan enterprise sesungguhnya

IN-MEMORY STATE & MOCK STORAGE

Lab ini menggunakan in-memory map dan sync.Mutex bukan SQL/NoSQL database nyata
State machine orchestrator tidak disimpan di persistent storage disk
Jika proses Go di-kill di tengah eksekusi, in-flight saga state akan hilang

SIMPLIFIKASI JARINGAN

Pemanggilan antar service dilakukan secara in-process function call, bukan gRPC/HTTP
EventBus menggunakan Go channel/slice in-memory, bukan Apache Kafka atau RabbitMQ
Tidak menyimulasikan network partition nyata, tetapi memverifikasi logika koordinasi

Algoritma kompensasi LIFO terbukti secara matematis dan logis

Mekanisme idempotency key dan semantic lock diverifikasi 100% fungsional

Fondasi konseptual yang kokoh sebelum mengadopsi engine seperti Temporal

This lab proves algorithmic correctness in-memory; production requires persistent state engines.

Tiga Prinsip Inti Transaksi Terdistribusi

Pedoman mendasar bagi software engineer dalam mendesain sistem microservices

PRINSIP 1: COMMIT LOKAL, KOMPENSASI GLOBAL

Jangan pernah mencoba mengunci database jarak jauh secara sinkron
Lakukan commit cepat di batas lokal dan andalkan kompensasi jika gagal
Maksimumkan ketersediaan sistem dan ketahanan terhadap partisi jaringan

PRINSIP 2: IDEMPOTENSI ADALAH HARGA MATI

Dalam sistem terdistribusi, kegagalan jaringan tidak terhindarkan
Retry yang aman hanya dapat terwujud jika setiap langkah memiliki idempotency key
Desain fungsi agar dapat dipanggil berkali-kali dengan hasil yang sama persis

Ketiadaan isolasi ACID harus diimbangi oleh semantic locking di level aplikasi

Tandai status data PENDING agar modifikasi konkuren tidak merusak integritas

Lindungi sistem dari anomali data dirty reads dan lost updates

Three rules of sagas: commit locally, make every step idempotent, and lock semantically.

Ringkasan Lab 29: Saga Pattern

Sagas trade strict ACID isolation for resilient, scalable eventual consistency across microservices.

HASIL VERIFIKASI

Happy path orkestrasi: 4 langkah tuntas dengan status APPROVED

Rollback kompensasi LIFO: Berhasil refund payment dan cancel order saat inventaris gagal

9 unit, concurrency, dan cancellation test suites lulus sempurna (go test -v -race)

REPOSITORY

https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/29-saga-pattern

Lukman / lukman-ss

MENTAL MODEL UTAMA

A saga is a sequence of local transactions where each step commits immediately.

If any step fails, compensating transactions execute in reverse order (LIFO).

Eventual consistency replaces strict 2PC locking across service boundaries.

RESOURCES & AUDIT

Repository: https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/29-saga-pattern

Commit SHA: 9c525944dc2de00e16513830a25ffa1c4834d742

Jalankan Demo: go run ./cmd/demo (di folder lab)

Jalankan Test: go test -v -race ./... (di folder lab)